WIPRO SDET TRAINING PROGRAM

Capstone Project Report

Frontend Automation Testing
using Playwright

Application Under Test: **blinkit**

blinkit.com · Quick-commerce Grocery Delivery

By **Aryan Mohanty**

Introduction

1.1 Objective

The objective of this capstone project is to design and develop a robust end-to-end automation testing framework using Playwright as part of the Wipro SDET Training Program. The framework automates critical user journeys on a real-world quick-commerce platform, validates UI functionality across multiple application modules, and generates detailed execution reports using Allure.

The project demonstrates practical application of industry-standard automation practices including the Page Object Model, fixture-based test setup, cross-browser execution, and structured test design.

1.2 Scope

The project covers frontend automation testing of the Blinkit web application across 9 functional modules with a total of 121 test cases. Testing is limited to the frontend UI layer no backend APIs, databases, or real financial transactions are involved.

The framework is validated across three browser engines — ***Chromium***, ***Firefox***, *and* ***WebKit*** ensuring consistent behavior across different environments.

1.3 Tools & Technologies

The following tools and technologies were used to build and execute the automation framework for this project:

Tool	Purpose
Playwright	Core automation framework
JavaScript	Test scripting language
Node.js	Runtime environment
Allure	Test reporting and analytics
VS Code	Development environment
GitHub	Version control

Application Under Test: blinkit

Blinkit is a leading quick-commerce grocery delivery platform operating across major cities in India. The platform enables users to browse products, manage carts, place orders, and track deliveries through a modern, responsive web interface built on a production-grade frontend architecture.

The platform maintains consistent uptime and availability as a live production environment, ensuring test cases can be executed repeatedly without disruption. Its rich set of real-world workflows and dynamic UI components make it an excellent choice for practising and validating Playwright-specific capabilities.

Key Reasons for Choosing:

- Stable production environment with consistent uptime, minimising test disruption from server outages
- Real-world e-commerce workflows including authentication, product search, cart management, and checkout navigation
- Dynamic frontend components ideal for validating Playwright locators, auto-waiting, and synchronisation handling
- Modern responsive UI with reusable DOM structures suitable for scalable Page Object Model design
- Rich feature coverage enabling multiple automation modules across the full user journey
- Publicly accessible platform requiring no backend access or real financial transactions

MODULES	TEST CASES COVERED
9	121
Browser	3
Reporting Tool	Allure
Language	JavaScript

MODULE	TEST CASE	TEST CASES
Homepage & Navigation	Homepage loading, navigation menu, banners, search bar, footer links, responsive homepage UI	15
Search Functionality	Product search, invalid search, search suggestions	13
Product Listing	Category pages, filters, sorting, product cards	13
Product Details	Product info, images, specifications, quantity selection, wishlist	10
Cart Management	Add/remove products, quantity updates, subtotal validation, cart persistence	18
Checkout Flow	Checkout navigation, address validation, add new address flow, authentication gate for checkout access	10
Authentication & User Account	Login, invalid credentials, session persistence, account validation, OTP validation	15
Store Functionalities	Validates store navigation, category selection, product availability, and store-specific interactions	15
Responsive & Cross-Browser UI	Mobile UI, tablet UI, hamburger menu, Firefox/Edge validation	13

Priority	P0 *(Critical core flows (homepage, search, cart, login)*	P1 *(Important navigation & UI behavior features)*	P2 *(Low priority UI polish and edge cases)*

MODULE 1 | HOMEPAGE & NAVIGATION (HN)

The Homepage & Navigation module validates the core entry point of the Blinkit web application. This module ensures that the homepage loads correctly, all key UI elements are visible and interactive, and navigation flows work as expected across both desktop and mobile viewports.

It covers essential user interactions such as cookie acceptance, search initiation, cart access, logo navigation, footer links, sticky navbar behaviour, location popup, and mobile responsiveness — forming the foundation for all other modules.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
HN_01	Verify homepage loads successfully	Homepage should load without errors and all key UI elements should be visible	P0
HN_02	Verify location popup acceptance	Location popup should disappear after entering valid location	P0
HN_03	Verify Shop Now button navigation	User should be redirected to correct product or purchase page	P0
HN_04	Verify Logo returns to Homepage	Verify that clicking the Logo redirects the user back to the Homepage successfully	P0
HN_05	Verify search box interaction	Search input field should open on clicking search icon	P0
HN_06	Verify search input accepts typing	User should be able to type without lag or errors	P0
HN_07	Verify search and redirect functionality	Search results page should load based on query	P0
HN_08	Verify cart opens	Cart sidebar/page should open successfully	P0
HN_09	Verify footer	Footer should be displayed correctly without UI issues	P0
HN_10	Verify footer contact link	Contact page should load successfully without any errors	P2

HN_11	Verify mobile menu	Hamburger menu should open and show navigation links	P0
HN_12	Verify sticky navbar	Navbar should remain visible during scroll	P0
HN_13	Verify mobile responsiveness	The application should be fully responsive on mobile view	P0
HN_14	Verify search suggestion appear	Search suggestions should appear correctly based on input	P0
HN_15	Verify homepage refresh behavior	Homepage should reload correctly without breaking UI or data	P0

This module confirms that the homepage and navigation layer of the application is functionally stable, visually consistent, and responsive across devices. It ensures that all primary entry-level interactions are working as expected, providing a solid base for downstream modules such as product search, cart management, and checkout flows.

MODULE 2 | SEARCH FUNCTIONALITY (SF)

The Search Functionality module validates the core product discovery feature of the Blinkit web application. This module ensures that users can efficiently search for products, view relevant results, and interact with the search interface without errors or performance issues.

It covers essential scenarios such as opening the search input, entering search queries, validating real-time input behavior, verifying search suggestions, ensuring accurate redirection to the results page, and confirming that navigation back to previous states works as expected.

This module is critical for user experience as search is one of the primary ways users discover products within the application, and it ensures that the search system is responsive, accurate, and stable across different devices and viewports.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
S_01	Verify search icon opens search input field	Search textbox should be displayed when search icon is clicked	P0
S_02	Verify search textbox accepts user input	User should be able to type text without errors	P0
S_03	Verify search redirects to results page	Valid search should redirect to relevant results page	P0
S_04	Verify invalid search handling	No results message should be shown for invalid queries	P1
S_05	Verify empty search validation	System should handle empty search gracefully (no crash / proper message)	P0
S_06	Verify multiple consecutive searches	User should be able to perform multiple searches without issues	P1
S_07	Verify search close functionality	Search panel should close successfully when closed	P1
S_08	Verify search works in mobile viewport	Search functionality should work correctly in mobile layout	P0
S_09	Verify search with uppercase input	Search should return correct results regardless of letter case	P1
S_10	Verify product navigation from search results	User should be able to open product page from search results	P0
S_11	Verify special character search handling	The application should handle special characters gracefully without breaking UI or functionality	P1
S_12	Verify search suggestions appear dynamically	Search suggestions should appear dynamically as the user types	P1
S_13	Verify search results display correctly	Search results page should load correctly with relevant products displayed	P0

MODULE 3 | PRODUCT LISTING (PL)

This module validates the Product Listing Page (PL) of the Blinkit web application. It ensures that products are correctly displayed after location selection and that users can interact with product cards seamlessly.

The module covers key functionalities such as product visibility, product card rendering, image loading, price display, navigation to product detail pages, add-to-cart functionality, scrolling behavior, and consistency of product listings across refresh and navigation actions.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
PL_01	Verify product listing page loads correctly	Product listing page should load with all products visible	P0
PL_02	Verify product page opens correctly	Clicking a product should open the correct product details page	P0
PL_03	Verify Add Now button works correctly	Buy Now button should redirect user to purchase/checkout flow	P0
PL_04	Verify Learn More button works correctly	Learn More button should open product details/info section	P1
PL_05	Verify product images load correctly	All product images should load without errors	P0
PL_06	Verify product price visibility	Each product displays correct price	P0
PL_07	Verify product cart contains image	Product image should be visible inside cart	P1
PL_08	Verify multiple product cards displayed	Multiple product cards should be rendered on the listing page	P0
PL_09	Verify product listing available after refresh	Page should reload and maintain product listing without errors	P1
PL_10	Verify user can return to product listing from product details	Back navigation should return user to product listing page	P1
PL_11	Verify multiple product images displayed	Each product card should display at least one image	P1
PL_12	Verify product remains visible after scrolling	Products should stay visible and load correctly on scroll	P1
PL_13	Verify at least 5 product cards displayed	Minimum 5 product cards should be visible on the listing page	P1

MODULE 4 | PRODUCT DETAILS (PD)

The Product Details module validates the Product Detail Page (PDP) of the Blinkit web application. This module ensures that users can view complete product information after selecting an item from the product listing page, including product name, images, pricing, and related details.

It covers essential functionalities such as navigation from Product Listing Page (PL) to Product Detail Page (PD), verification of product information visibility, image rendering, and consistency of data displayed for selected products. It also ensures that users can seamlessly return to the listing page without losing context or encountering UI issues.

This module is critical for validating the accuracy and completeness of product-level information, ensuring users have a clear and reliable view of selected items before making purchase decisions.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
PD_01	Verify product details page loads correctly	Product details page should load without errors	P0
PD_02	Verify product information is displayed correctly	Product name, description, and price should be visible	P0
PD_03	Verify product images load correctly	Product images should display properly without broken images	P0
PD_04	Verify image gallery functionality	Product image gallery should be visible	P1
PD_05	Verify product specifications are visible	Product specifications/details section should display correctly	P1
PD_06	Verify quantity selection functionality	User should be able to increase or decrease product quantity	P0
PD_07	Verify quantity cannot go below minimum	System should not allow quantity to go below minimum value (1)	P1
PD_08	Verify Add to Cart button functionality	Product should be added to cart successfully	P0
PD_09	Verify 'Why shop from blinkit' section visibility	The section should be visible and correctly rendered on PD	P2
PD_10	Verify product availability status	Product stock/availability information should be displayed correctly	P1
PD_11	Verify product Mobile Responsiveness	Product details page should adapt correctly across mobile and desktop views	P0

MODULE 5 | CART FUNCTIONALITY (CT)

The Cart Functionality module validates the shopping cart behavior of the Blinkit web application. This module ensures that users can successfully add products to the cart, view cart contents, and manage selected items without errors or inconsistencies.

It covers essential workflows such as adding products from the product listing and product detail pages, verifying cart updates, checking item quantity changes, and ensuring accurate cart state persistence during navigation and refresh actions. It also validates that the cart interface remains responsive and consistent across different devices and viewports.

This module is critical for ensuring a smooth purchase flow, as the cart acts as a central component between product selection and checkout, directly impacting the user's buying experience.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
CT_01	Verify Cart Visibility	Cart icon/section should be visible on the page	P0
CT_02	Verify Add to Cart Functionality	User should be able to add products to cart successfully	P0
CT_03	Verify sidebar cart opens	Clicking cart icon should open cart drawer/sidebar	P0
CT_04	Verify added product appears in cart	Added product should be visible inside the cart	P0
CT_05	Verify product quantity increment	Increasing quantity should update product count correctly	P0
CT_06	Verify product quantity decrement	Decreasing quantity should reduce product count correctly	P0
CT_07	Verify product removal from cart	Removing product should delete it from cart successfully	P0
CT_08	Verify cart total price updates dynamically	Cart total should update correctly when quantity changes	P0
CT_09	Verify multiple products can be added to cart	User should be able to add more than one product to cart	P0
CT_10	Verify same product quantity increases instead of duplicate entry	Adding same product should increase quantity instead of creating duplicate entries	P0
CT_11	Verify cart count badge updates correctly	Cart icon badge should reflect correct number of items	P0

CT_12	Verify cart is empty state	Cart should display empty state when no items are present	P1
CT_13	Verify cart validation	System should prevent invalid cart operations	P1
CT_14	Verify cart persists across navigation	Cart items should remain after navigating between pages	P0
CT_15	Verify cart drawer closes	Cart sidebar should close successfully when close button is clicked	P1
CT_16	Verify clicking outside closes cart drawer	Clicking outside cart should close the cart sidebar	P1
CT_17	Verify max quantity handling	System should restrict quantity beyond allowed limit	P2
CT_18	Verify cart total updates after quantity increase	Cart total should increase when quantity is increased	P0
CT_19	Verify cart total decreases after quantity reduction	Cart total should decrease when quantity is reduced	P0

This module ensures that the cart functionality operates reliably throughout the shopping journey by validating product addition, quantity management, cart updates, and item removal workflows. It confirms that cart data remains accurate and consistent across user interactions, providing a seamless transition from product selection to checkout while maintaining a stable and user-friendly experience.

MODULE 6 | Checkout Flow (CF)

The Checkout Flow module validates the transition from cart management to order fulfillment within the Blinkit web application. This module ensures that users can successfully proceed through the checkout process, access delivery options, and manage address information required for order placement.It covers critical checkout scenarios such as navigating from the cart to checkout, validating checkout access restrictions, verifying user authentication requirements, selecting saved delivery addresses, and adding new delivery addresses during the checkout process. The module also validates checkout page elements, address forms, and navigation continuity between cart and checkout screens.

As the final stage before order confirmation, this module plays a crucial role in ensuring a seamless purchasing experience by verifying that all prerequisite checkout workflows function reliably and securely.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
CF_01	Verify user can navigate to checkout from cart	User should be able to proceed from cart to checkout flow successfully	P0
CF_02	Verify checkout is inaccessible with empty cart	User should not be allowed to access checkout without items in cart	P0
CF_03	Verify navigation back to cart from checkout	User should be able to return to cart without losing added items	P1
CF_04	Verify checkout requires user authentication	Unauthenticated users should be redirected away from checkout page	P0
CF_05	Verify checkout progress section visibility	Checkout page should display progress and order summary information correctly	P1
CF_06	Verify saved delivery address selection	User should be able to select an existing saved delivery address	P0
CF_07	Verify Add New Address option availability	User should be able to access the Add New Address functionality during checkout	P1
CF_08	Verify user can initiate address creation	Add New Address flow should open successfully from checkout	P1
CF_09	Verify new address form visibility	Address creation form should be displayed when Add New	P1
CF_10	Verify required address fields are displayed	Address form should contain all mandatory input fields required for address creation	P0

This module ensures that the checkout workflow functions correctly by validating cart-to-checkout navigation, user authentication requirements, address management, and checkout page accessibility. It confirms that users can seamlessly proceed toward order placement while maintaining a consistent and reliable checkout experience.

MODULE 7 | AUTHENTICATION MANAGEMENT (AT)

The Authentication & Session Management module validates user access control, login workflows, and session persistence within the Blinkit web application. This module ensures that users can initiate the login process, interact with phone number and OTP verification screens, and access authenticated features after successful login.

It covers critical authentication scenarios such as login popup visibility, phone number validation, OTP verification flow, resend OTP functionality, navigation between authentication screens, and login dialog management. The module also validates authenticated user behavior including account access, session persistence, profile navigation, and account-related functionality.

To improve automation reliability and execution speed, authenticated test scenarios utilize a pre-generated authentication state (auth.json) created through a dedicated authentication setup script. This approach enables validation of post-login functionality without requiring repeated OTP verification during every test execution.

This module ensures that user authentication, authorization, and session management mechanisms function reliably while providing a seamless and secure user experience

Table 1: Login Flow Tests

TEST CASE ID	DESCRIPTION	Expected Result	Priority
AT_01	Verify login popup opens successfully	Login popup should open and display phone number input field	P0
AT_02	Verify phone number field accepts valid input	User should be able to enter a valid phone number	P0
AT_03	Verify phone number validation for invalid input	System should handle invalid phone number input appropriately	P1
AT_04	Verify OTP screen appears after valid phone number	OTP verification screen should be displayed after submitting a valid phone number	P0
AT_05	Verify OTP fields accept numeric input	OTP input fields should accept numeric digits correctly	P1
AT_06	Verify Resend OTP option visibility	Resend OTP option should be displayed on OTP verification screen	P1
AT_07	Verify Resend OTP functionality	User should be able to trigger OTP resend action	P1
AT_08	Verify back navigation from OTP screen	User should be returned to phone number entry screen	P1

AT_09	Verify complete OTP entry functionality	User should be able to enter all OTP digits successfully	P1
AT_10	Verify login popup close functionality	Login popup should close successfully when dismissed	P1

Table 2: Authenticated Session Tests

TEST CASE ID	DESCRIPTION	Expected Result	Priority
AT_11	Verify user remains logged in	Authenticated user should remain logged in and Account option should be visible	P0
AT_12	Verify account menu opens successfully	Account menu should open and display account-related options	P0
AT_13	Verify session persistence after refresh	User session should remain active after page refresh	P0
AT_14	Verify My Orders navigation	User should be redirected to the Orders page successfully	P1
AT_15	Verify Saved Addresses navigation	User should be redirected to the Saved Addresses page successfully	P1

This module ensures that authentication and session management features operate reliably by validating login workflows, OTP verification, user access controls, and authenticated account functionality. It confirms that users can securely access protected features while maintaining a consistent session experience across navigation and page refresh actions.

Authentication-dependent test cases utilize a pre-generated authentication state (`auth.json`) created through a dedicated setup script. Sensitive credentials and user-specific data are managed through environment variables (`.env`) and are excluded from source control for security purposes.

MODULE 8 | STORE NAVIGATION (SN)

The Store & Category Navigation module validates the category browsing experience of the Blinkit web application. This module ensures that users can navigate through different product categories, access category-specific product listings, and interact with store pages without encountering navigation or content display issues.

It covers key functionalities such as category selection, category page loading, URL validation, product visibility, image rendering, price display, category persistence after refresh, and scrolling behavior. The module also verifies that category pages contain valid product data and maintain a consistent browsing experience across multiple store sections.

This module is essential for validating product discovery workflows, ensuring that users can efficiently browse categories and access relevant products through the store navigation system.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
SN_01	Verify Category-1 Navigation works	User should be redirected to the correct Cold Drinks & Juices category page	P0
SN_02	Verify Category-2 Navigation works	User should be redirected to the correct Dairy category page	P0
SN_03	Verify Category-3 Navigation works	User should be redirected to the correct Fruits & Vegetables category page	P0
SN_04	Verify category page displays products	Selected category page should display category heading and products successfully	P0
SN_05	Verify product price visibility on category page	Product prices should be visible on category page	P0
SN_06	Verify product image visibility on category page	Product images should be displayed correctly without broken images	P0
SN_07	Verify category page is not empty	Category page should contain one or more products	P0
SN_08	Verify category URL structure	Category page URL should follow the expected category URL format	P1

SN_09	Verify category page loads successfully	Category page should load successfully without UI or navigation errors	P0
SN_10	Verify category page persistence after refresh	Category page should remain accessible after page refresh	P1
SN_11	Verify scrolling through category page	User should be able to scroll through category page without content issues	P1
SN_12	Verify product availability within category	Category page should contain visible and accessible products	P0

This module ensures that category browsing and store navigation features function reliably by validating category access, product visibility, page loading behavior, and navigation consistency. It confirms that users can efficiently browse products across different categories while maintaining a smooth and uninterrupted product discovery experience.

MODULE 9 | MOBILE RESPONSIVENESS & INTERACTION (MI)

The Mobile Responsiveness & Interaction module validates the behavior, usability, and accessibility of the Blinkit web application across mobile devices and tablet viewports. This module ensures that the application adapts correctly to different screen sizes while maintaining core functionality and a seamless user experience.

It covers key mobile-specific scenarios such as application loading, location selection, category visibility, search accessibility, cart visibility, mobile navigation elements, scrolling behavior, logo rendering, and responsive layout validation across multiple device profiles. The module also verifies that essential user journeys remain functional on smartphones and tablets without layout distortions or usability issues. This module is critical for ensuring that users can effectively interact with the application on mobile devices, providing a consistent and responsive shopping experience across various screen resolutions and device types.

TEST CASE ID	DESCRIPTION	Expected Result	Priority
MI_01	Verify application loads on mobile viewport	Application should load successfully on mobile device viewport	P0
MI_02	Verify intro slider can be closed	User should be able to close the introductory slider successfully	P1

MI_03	Verify delivery location selection on mobile	User should be able to select a delivery location successfully	P0
MI_04	Verify search interface visibility	Search functionality should be accessible after location selection	P0
MI_05	Verify category visibility on mobile	Product categories should be visible and accessible on mobile layout	P0
MI_06	Verify mobile navigation menu interaction	Mobile navigation menu should open successfully when available	P2
MI_07	Verify cart visibility on mobile	Cart section should be visible after adding a product	P0
MI_08	Verify application loads across multiple mobile devices	Application should load correctly on supported mobile and tablet devices	P1
MI_09	Verify search bar visibility across devices	Search bar should be displayed correctly on different device viewports	P1
MI_10	Verify Blinkit logo visibility on mobile	Blinkit logo should be displayed correctly on mobile screens	P2
MI_11	Verify product page accessibility on mobile	Product listing/search page should load correctly on mobile viewport	P0
MI_12	Verify page scrolling on mobile	User should be able to scroll through the page without issues	P1
MI_13	Verify page title on mobile	Correct application title should be displayed on mobile devices	P2

This module ensures that the Blinkit application delivers a consistent and responsive experience across mobile and tablet devices by validating layout adaptation, navigation accessibility, search functionality, cart interactions, and general usability. It confirms that critical user workflows remain functional and visually stable across various viewport sizes, supporting a seamless mobile shopping experience.

TEST REPORTING & EXECUTION RESULTS

To evaluate the effectiveness and reliability of the automation framework, test execution results were generated using the Allure Reporting Framework integrated with Playwright. Allure provides detailed execution insights including test status, execution trends, duration analysis, browser-wise execution results, and failure diagnostics.

The automation suite was executed across three major browser engines supported by Playwright:

- ***Chromium***
- ***Firefox***
- ***WebKit***

A total of **121** automated test cases were executed during the final test cycle. The majority of test cases passed successfully across all supported browsers, validating the stability of the automation framework and the core functionality of the application.

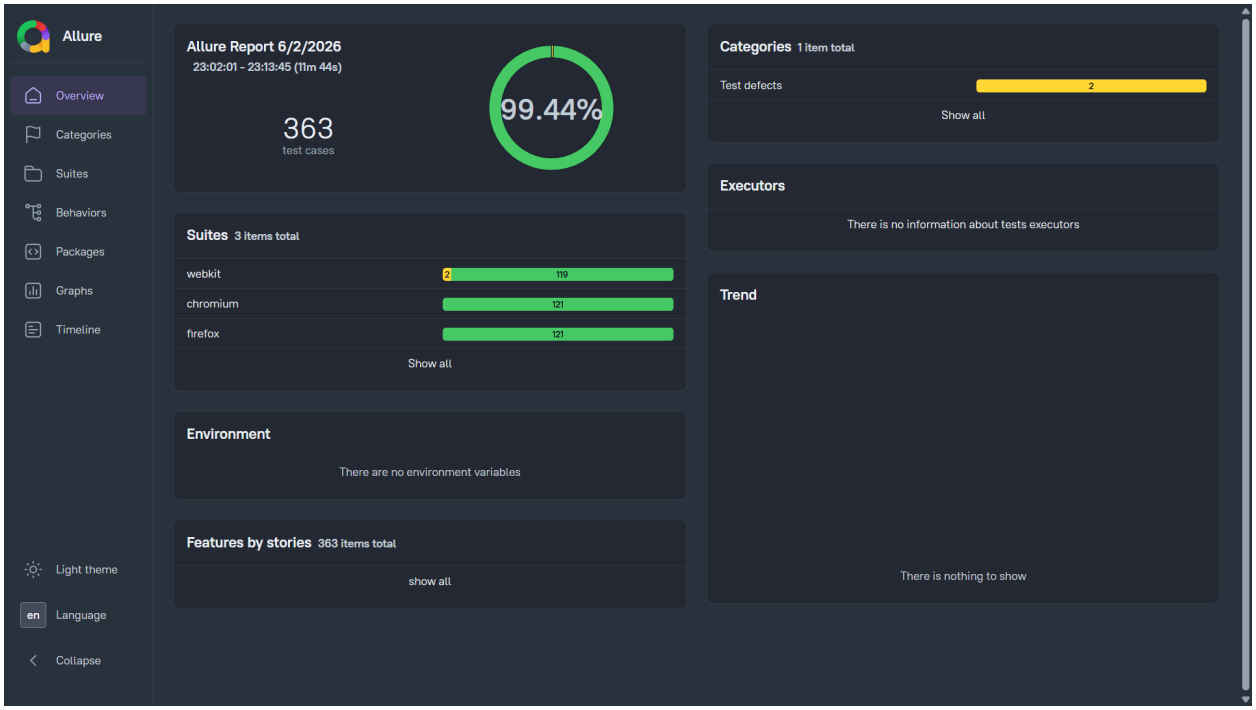

Figure 1: Allure Report Dashboard Overview

The Allure dashboard provides a consolidated view of the automation execution results across Chromium, Firefox, and WebKit browsers. A total of 363 executions were recorded (121 test cases executed across three browser engines), achieving an overall pass rate of 99.44%.

Browser execution results are summarized below:

Browser	Total Executed	Passed	Failed	Flaky
Chromium	121	121	0	0
Firefox	121	119	0	2
WebKit	121	121	0	0

Figure 2: Allure Execution Graph

The Allure execution graph provides a visual representation of the overall test execution results across all supported browser environments. The graph highlights the distribution of passed, failed, and flaky test cases, enabling quick assessment of automation suite stability and execution quality.

The two flaky test cases observed during Firefox execution were caused by browser-specific timing and synchronization issues, resulting in intermittent timeout failures. These failures were not reproducible consistently and did not indicate application defects. Such behavior is common in UI automation when dealing with dynamic web elements and varying browser rendering speeds. The Allure report provided detailed visibility into execution status, execution duration, browser coverage, and individual test case outcomes, enabling efficient analysis and validation of the automated test suite.

TEST LIMITATIONS AND EXCLUSIONS

During automation development, several potential test scenarios were evaluated but not included in the final execution suite due to application-specific UI challenges that affected automation reliability.

The primary challenges encountered included:

• Dynamic and frequently changing locators across application updates
• Inconsistent button labels and element naming conventions
• Elements with unstable visibility states during page loading
• Components rendered conditionally based on user interactions or location selection
• Browser-specific timing and synchronization issues affecting element detection
• UI elements becoming detached or re-rendered during execution

To improve automation stability and maintainability, reusable Page Object Model (POM) components and custom Playwright fixtures were implemented wherever applicable. These practices helped reduce code duplication, improve locator management, and provide more reliable test execution across multiple test modules and browser environments.

Despite these improvements, a small number of scenarios remained highly dependent on unstable UI behavior and dynamic application rendering. Such scenarios were excluded from the final execution suite to minimize false failures and ensure that reported results accurately reflected application quality rather than automation instability.

The final automation suite focuses on stable, repeatable, and business-critical user journeys while maintaining comprehensive coverage of the application's core functionality.

CHALLENGES FACED

During the automation of the Blinkit web application, several challenges were encountered that impacted test design, locator strategy, and execution stability.

1. ***Dynamic and Unstable Locators***
   Many UI elements were generated using dynamic class names and frequently changing attributes, making locator identification challenging. Several locators became invalid after UI updates, requiring frequent maintenance and refinement.
2. ***Non-Descriptive Button Labels***
   Certain interactive elements used unconventional labels and symbols instead of meaningful text. Examples included numeric values being used for quantity increment actions and symbols replacing standard increment/decrement controls. This increased the complexity of creating stable and readable automation scripts.

3. ***Image-Based Navigation Components***
   Several navigation elements relied primarily on images or icons rather than unique IDs, data attributes, or descriptive element names. This required the use of alternative locator strategies such as image alt text, role-based locators, and relative element identification.
4. ***Browser Synchronization Issues***
   Some test scenarios exhibited timing-related inconsistencies across different browser engines, particularly Firefox. Elements occasionally required additional synchronization handling due to delayed rendering and dynamic content loading.
5. ***Responsive UI Variations***
   The application displayed different layouts and interaction patterns across desktop, mobile, and tablet viewports. Additional validation and device-specific handling were required to ensure consistent automation coverage across supported screen sizes.
6. ***Conditional Rendering of Elements***
   Certain components were rendered only after specific user interactions, location selections, or page state changes. This required the implementation of explicit waits and conditional checks to improve execution reliability.

Despite these challenges, the adoption of Playwright best practices, reusable fixtures, and the Page Object Model (POM) approach helped improve script maintainability, reduce duplication, and enhance overall test stability.

CONCLUSION

Building an automation framework against a live production application like Blinkit turned out to be a fundamentally different challenge than testing against a controlled environment. Dynamic locators, conditional rendering, OTP-gated authentication, and browser-specific timing issues forced real engineering decisions — not just test scripting.

The final framework covers 9 functional modules with 121 automated test cases executed across Chromium, Firefox, and WebKit, achieving a 99.44% pass rate. Practices like the Page Object Model, fixture-based setup, and pre-generated authentication state (auth.json) were adopted not for the sake of it, but because the application genuinely demanded them.

The workflow file for CI/CD integration via GitHub Actions is already scaffolded in the repository — pipeline execution was attempted but not fully deployed due to configuration constraints during the training period. API-layer testing and accessibility validation would be the natural next extensions, covering what frontend UI tests inherently cannot catch.

This project solidified one thing clearly: automation quality is only as good as the decisions made before writing the first test.

GITHUB REPOSITORY: https://github.com/AryanMohanty04/playwright-capstone-project